1.4 API First: Integracja z OpenAI API i Anthropic API (autoryzacja, koszty, parametry temperature/top_p)

Moduł 1: Fundamenty i Mechanizmy LLM

Krzysztof Rytlewski · 2026

Podejście API First

API First oznacza traktowanie interfejsu programistycznego jako produktu pierwszej klasy — jest projektowany i dokumentowany przed implementacją pozostałych warstw systemu.

W kontekście LLM to świadome odseparowanie modelu (inteligencji) od aplikacji i komunikacja wyłącznie przez dobrze zdefiniowany kontrakt HTTP. Zmiana dostawcy (GPT-4o → Claude) powinna wymagać korekty kilku linii kodu, nie przepisania aplikacji. Dobra warstwa abstrakcji nad API LLM to różnica między projektem, który można utrzymywać, a projektem nie do ruszenia po roku.

Kluczowe pojęcia:

- **Endpoint** — adres URL przyjmujący żądania (np. `https://api.openai.com/v1/chat/completions`)

- **Rate limit** — limit żądań lub tokenów na jednostkę czasu, nakładany przez dostawcę

- **Zasada bezpieczeństwa** — klucze API nigdy nie opuszczają backendu; wywołanie z przeglądarki naraża klucz na kradzież przez DevTools

Autoryzacja — klucze API i dobre praktyki bezpieczeństwa

Zarówno OpenAI, jak i Anthropic używają Bearer token authentication — klucz jest przekazywany w nagłówku HTTP przy każdym żądaniu.

Dostawca	Nagłówek autoryzacji	Dodatkowe nagłówki
OpenAI	`Authorization: Bearer sk-...`	(opcjonalnie) `OpenAI-Organization`
Anthropic	`x-api-key: sk-ant-...`	`anthropic-version: 2023-06-01` (wymagany)

Dobre praktyki przechowywania kluczy:

- **Dev/staging** — zmienne środowiskowe w pliku `.env` (nigdy nie commituj do repozytorium; dodaj do `.gitignore`)
- **Produkcja (AWS)** — AWS Secrets Manager lub AWS Parameter Store, nigdy w `docker-compose.yml` ani w kodzie
- Twórz osobne klucze dla każdego środowiska (dev/staging/prod) i rotuj je regularnie

Wywołanie OpenAI Chat Completions API

Minimalne wywołanie OpenAI Chat Completions API w Node.js — wiadomości przesyłane są jako tablica z rolami `system`, `user` i `assistant`:

```
const response = await openai.chat.completions.create({
  model: 'gpt-4o',
  messages: [
    { role: 'system', content: 'Jesteś pomocnym asystentem programisty.' },
    { role: 'user',   content: 'Wyjaśnij różnicę między Promise a async/await.' }
  ],
  max_tokens: 512,
  temperature: 0.7,
});
console.log(response.choices[0].message.content);
console.log(response.usage); // { prompt_tokens, completion_tokens, total_tokens }
```

Wywołanie Anthropic Messages API

Anthropic API różni się strukturą — system prompt jest osobnym polem na najwyższym poziomie żądania:

```
const response = await anthropic.messages.create({
  model: 'claude-3-5-sonnet-20241022',
  max_tokens: 512,
  system: 'Jesteś pomocnym asystentem programisty.',
  messages: [
    { role: 'user', content: 'Wyjaśnij różnicę między Promise a async/await.' }
  ],
});
console.log(response.content[0].text);
console.log(response.usage); // { input_tokens, output_tokens }
```

Ważne: wiadomości w historii muszą być **naprzemienne** (user → assistant → user...). Dwie wiadomości `user` pod rząd spowodują błąd walidacji w API Anthropic.

Kluczowe różnice strukturalne między API

Aspekt	OpenAI	Anthropic
System prompt	Wiadomość `{ role: 'system', ... }`	Osobne pole `system: "..."`
Odpowiedź	`response.choices[0].message.content`	`response.content[0].text`
Tokeny wejściowe	`response.usage.prompt_tokens`	`response.usage.input_tokens`
Wymagany nagłówek	Brak	`anthropic-version` (zawsze)
Naprzemienność ról	Zalecana	Ściśle wymagana

Modele nie mają wbudowanej pamięci — to deweloper przechowuje historię konwersacji i przesyła ją przy każdym kolejnym wywołaniu API.

Parametry generowania — Temperature

Temperature skaluje rozkład prawdopodobieństwa nad słownikiem tokenów przed próbkowaniem:

- **temperature = 0.0** — zawsze wybiera token z najwyższym prawdopodobieństwem; zachowanie deterministyczne. Idealne do zadań faktograficznych, parsowania, generowania kodu

- **temperature ≈ 0.7** — złoty środek dla asystentów; odpowiedzi są konkretne, ale nie sztywne

- **temperature ≈ 1.2–2.0** — wysoka losowość, odpowiedź bywa nieprzewidywalna; używane w kreatywnym pisaniu i brainstormingu

Ważna zasada: nie używaj jednocześnie wysokiej temperatury i top_p — modele są kalibrowane do pracy z jednym z tych parametrów ustawionym blisko wartości domyślnej.

Top-p i tabela praktycznych ustawień

Top-p (nucleus sampling) — model bierze pod uwagę tylko tokeny odpowiadające za minimalny skumulowany zakres prawdopodobieństwa `p`. Przy `top_p = 0.9` odcinany jest ogon mało prawdopodobnych, często dziwnych tokenów.

Zastosowanie	temperature	top_p
Generowanie kodu	0.0–0.2	1.0
Ekstrakcja danych / JSON	0.0	1.0
Q&A / asystent	0.7	1.0
Copywriting	1.0	0.95
Kreatywne pisanie	1.2–1.5	0.9

Inne ważne parametry generowania

- `max_tokens` — twardy limit tokenów odpowiedzi; niezużyte tokeny nie są naliczane, ale limit nie gwarantuje konkretnej długości — ustaw go rozsądnie

- `stop` — lista ciągów znaków przerywających generowanie (np. `["###", "\n\n"]`); przydatne przy generowaniu strukturyzowanego tekstu

- `frequency_penalty` / `presence_penalty` (OpenAI) — kary za powtarzanie tokenów; zmniejszają tendencję modelu do zapętlania podobnych fraz

Dobieranie tych parametrów pozwala precyzyjnie kształtować wyjście modelu bez zmiany systemu promptów ani modelu bazowego.

Koszty — model pay-per-token i szacowanie

LLM API rozliczane są w modelu **pay-per-token** — osobna stawka za tokeny wejściowe (prompt) i wyjściowe (completion). Tokeny wyjściowe kosztują zazwyczaj **3–5× więcej** niż wejściowe.

Model	Wejście (/ 1M)	Wyjście (/ 1M)
GPT-4o	~2.50	10.00
GPT-4o mini	~0.15	0.60
Claude 3.5 Sonnet	~3.00	15.00
Claude 3 Haiku	~0.25	1.25

Monitoruj zużycie tokenów przy każdym wywołaniu (`response.usage`) i ustal limity budżetowe w dashboardach OpenAI/Anthropic.

Strategie redukcji kosztów

1. **Dobór modelu do zadania** — nie używaj GPT-4o do prostej klasyfikacji, gdy GPT-4o mini radzi sobie równie dobrze za 20× niższą cenę

2. **Skracanie system promptów** — każdy prompt powtarzany jest przy każdym wywołaniu; 500 zbędnych tokenów × 10 000 wywołań dziennie = 5 mln tokenów

3. **Prompt caching** — Anthropic i OpenAI oferują cachowanie powtarzalnych fragmentów kontekstu; cached tokeny kosztują ~10% normalnej ceny

4. **Ograniczanie `max_tokens`** — modele generują aż do limitu; ustaw rozsądny pułap dopasowany do zadania

5. **Streaming z przerwaniem** — generowanie można zatrzymać wcześniej, gdy użytkownik otrzymał wystarczające informacje

Obsługa błędów i retry z exponential backoff

Kod HTTP	Znaczenie	Działanie
`400`	Błędny format żądania	Sprawdź strukturę `messages`; nie ponów
`401`	Nieprawidłowy klucz API	Sprawdź zmienne środowiskowe; nie ponów
`429`	Przekroczono rate limit	Poczekaj i ponów z backoffem
`500 / 503`	Błąd po stronie dostawcy	Ponów z backoffem

Exponential backoff — przy błędach `429` i `5xx` ponów żądanie po opóźnieniu rosnącym wykładniczo (`1s`, `2s`, `4s`...) z dodaną losowością (*jitter*), aby unikać synchronizacji wielu klientów. Oficjalne SDK (`openai`, `@anthropic-ai/sdk`) mają tę logikę wbudowaną automatycznie.

Streaming — tokeny w czasie rzeczywistym

Streaming pozwala odbierać tokeny na bieżąco zamiast czekać na pełną odpowiedź. Poprawia UX i umożliwia wcześniejsze przerwanie generowania, co obniża koszty tokenów wyjściowych.

```
const stream = await openai.chat.completions.create({
  model: 'gpt-4o', messages, stream: true,
});
for await (const chunk of stream) {
  const delta = chunk.choices[0]?.delta?.content ?? '';
  process.stdout.write(delta); // lub push do WebSocket / SSE
}
```

Kluczowe wnioski — sekcja 1.4

- Klucze API przechowuj wyłącznie po stronie serwera — nigdy w kodzie klienta ani w repozytorium
- Zrozumienie `temperature` i `top_p` pozwala świadomie dobierać ustawienia do zadania: deterministyczność dla kodu, kreatywność dla pisania
- Model pay-per-token wymaga ciągłego monitorowania kosztów — tokeny wyjściowe kosztują 3–5× więcej niż wejściowe
- Warstwa abstrakcji nad API LLM (adapter) umożliwia przełączanie dostawców bez zmian w logice aplikacji
- Obsługa błędów z retry i backoffem oraz streaming to cechy dojrzałego, produkcyjnego kodu integrującego się z LLM